

CSC_4MI06 Rapport de Projet: Débruitage d'image par auto-encodeur

Tiago GENET, Henry ROYER

Objectifs

L'objectif de ce TP est d'étudier la capacité d'un réseau de neurones à éliminer ou à réduire le bruit dans une image. Pour construire notre réseau, nous partirons d'un code existant sous Keras (appuyé sur le backend TensorFlow) que nous allons, en premier lieu, entraîner sur la base de données phare du deep-learning: *MNIST*, qui contient des chiffres manuscrits. Notre objectif final consiste à appliquer notre modèle une fois entraîné sur des images bruitées de tailles arbitraires.

Ce rapport consiste en feuille de route détaillée sur les choix structurels de notre réseau et de son entraînement. L'entièreté du code utilisé se retrouve dans le notebook livrée avec ce rapport.

Sommaire

1	Auto-encodeur de base	2
2	Auto-encodeur pour le débruitage	3
3	Application de notre auto-encodeur à une image quelconque	5
4	Application de notre auto-encodeur sur de grandes photos	7



1 Auto-encodeur de base

Pour commencer ce projet on se contente d'abord d'exécuter et d'étudier la base de code fourni dans le notebook du projet.

Le notebook commence par importer tout les modules nécessaires à l'utilisation de Keras sur un backend par défaut (dans notre cas **tensorflow**) et il nous donne aussi quelques fonctions utiles pour travailler.

- **preprocess(array)**: Met en forme notre dataset d'images 28x28 en noir en blanc avec la valeur de chaque pixels normalisée sur $[0, 1]$.
- **noise(array, noise_factor)**: Ajoute un bruit aléatoire sur notre dataset selon le **noise_factor**.
- **display(array1, array2, nb)**: affiche un nombre **nb** d'images des deux datasets array1 & array2
- **show_learning_curves(history)**: affiche les courbes d'apprentissage pour les set d'entraînement et de validation pour notre auto-encodeur **history**

La structure de l'auto-encodeur pré implémenté à l'aide de l'API layers de Keras pour la représentation des images de MNIST est la suivante:

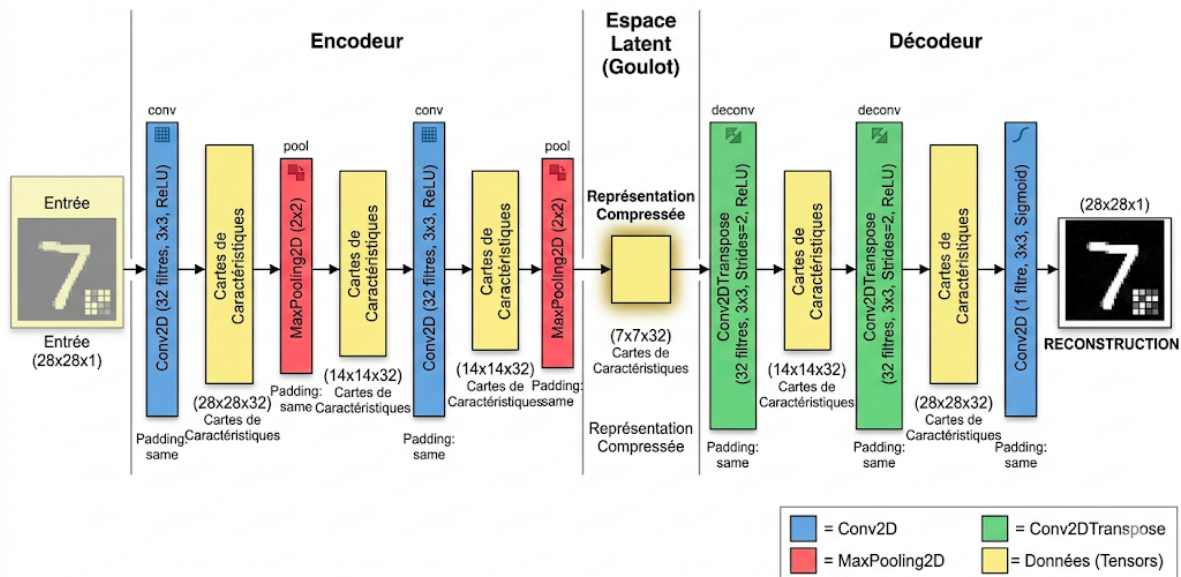


Figure 1: Visualisation des couches de notre auto-encodeur

On utilise un auto-encodeur à 4 couches d'encodage et 3 couches de décodage. Cela permet de passer par un espace latent composé de 32 cartes de caractéristiques 2D de dimension 7x7. Grâce à la méthode `.summary()` appliquée à notre auto-encodeur on se rend compte que ce petit réseau représente tout de même 28 353 paramètres au total !

Après un premier entraînement sur la base MNIST sur 6 epoch avec un `batch_size` de 80 on obtient des résultats visuellement très satisfaisants.

Toutefois même si les images semblent être les même en entrée/sortie de l'auto-encodeur ce n'est pas strictement le cas. Notre auto-encodeur apprend à reconstruire les images à partir de plus petits espaces de caractéristiques et tente de s'approcher au maximum des images que l'on lui fourni en entrée.

Or comme l'on peut le constater sur les exemples si-dessous, il y a toujours des légers écarts entre les images d'entrées et de sortie:



Figure 2: différences entre les images d'entrées (en haut) et les images reconstruites (en bas)

L'utilisation d'un auto-encodeur représente tout de même l'énorme avantage de pouvoir apprendre lui même des caractéristiques utiles à la reconstruction d'images du format MNIST de manière plus générales que sur notre seule base d'entraînement.

Puisque il apprend tout lui même il évite tout biais qui aurait pu être introduit si les caractéristiques à détecter avait été choisies par un humain spécifiquement. Il permet également, grâce à la partie encodage, de représenter nos images de manière compressée via la sélection de certaines cartes de caractéristiques.

Concernant la fonction loss choisie pour l'entraînement, dans notre cas la **binary cross entropy (BCE)**, il s'agit en effet pour notre application de la meilleure loss possible.

Plutôt que de voir le problème de reconstruction comme un problème de régression classique, la BCE traite plus le problème comme une classification entre si le pixel est blanc ou noir.

En effet dans notre traitement du dataset, toutes les images ont leurs valeurs de pixels normalisées sur $[0, 1]$ et la BCE est adaptée à cette vision probabiliste de la couleur d'un pixel. La formule de la BCE est la suivante :

$$L_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

Avec y_i la valeur réelle du pixel i et $\hat{y}_i$ la valeur prédite. On pénalise alors très fortement les énormes écarts de valeurs et très peu quand on approxime plutôt bien la luminosité d'un pixel.

Finalement on constate que notre auto-encodeur fourni des résultats plus que satisfaisant lorsque bien entraîné à reproduire des images à l'identique. Néanmoins une des application les plus courante des auto-encodeurs reste le débruitage d'images, aspect que nous explorons dans la partie suivante.

2 Auto-encodeur pour le débruitage

La deuxième partie de ce projet se concentre sur une des applications effective les plus courante des auto-encodeurs: le débruitage d'image.

On commence par utiliser notre fonction `noise` pour bruiteur notre dataset avec un premier facteur de bruit de 0,5 et on relance ensuite un nouvel entraînement avec en entrée nos images bruitées et en labels de sortie nos images originales. On obtient alors pour les mêmes paramètres d'entraînement (6 epoch, batch_size de 80) les résultats suivants :

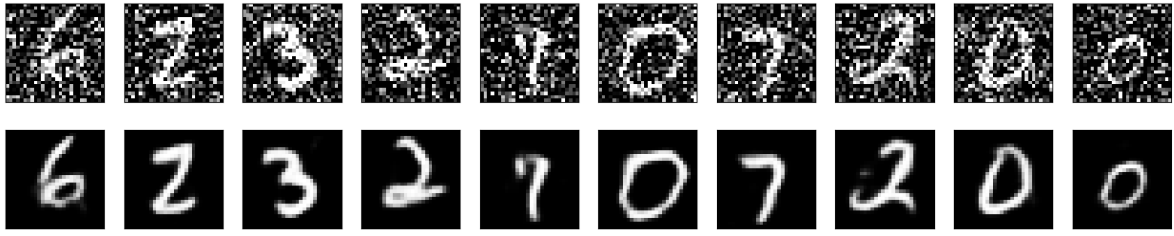


Figure 3: Résultats d'un débruitage pour un facteur de bruit de 0.5

Avec ces paramètres d'entraînement on atteint un niveau de loss en training et validation respectifs d'environ 10%

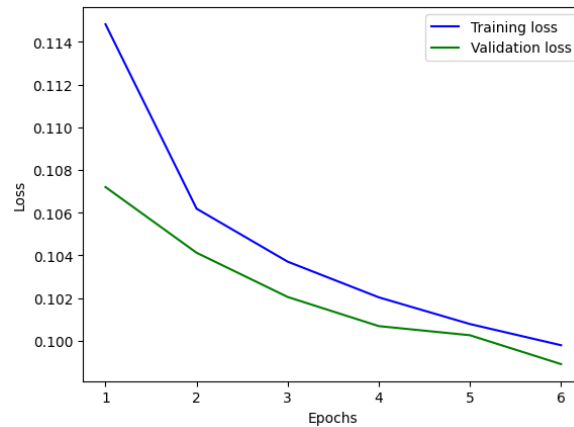


Figure 4: Courbes d'apprentissage sur l'entraînement au débruitage de notre réseau pour un niveau de bruit de 0,5

On relance ensuite plusieurs entraînements pour différents niveaux de bruits de plus en plus élevées pour tester les limites de notre auto-encodeur.

Visuellement on constate que notre auto-encodeur devient de plus en plus mauvais avec l'augmentation du facteur de bruit. Pour un facteur $\eta = 0.9$ on remarque visuellement que les images sont plus floues et que les représentations sont plus aléatoires.

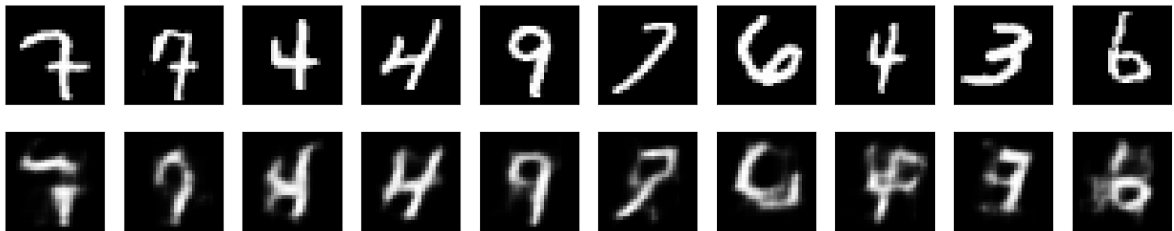
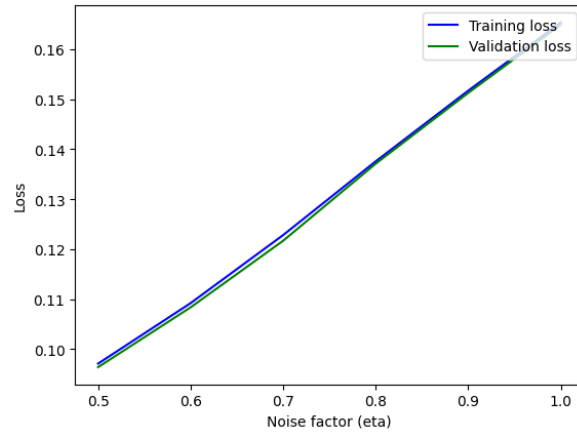


Figure 5: Résultat de débruitage pour un niveau de bruit de 0.9

Pour plus de rigueur on peut tracer la courbes de loss obtenue en test et validation en fonction de notre facteur de bruit entre 0,5 et 1. En utilisant toujours comme métrique de performance notre BCE, on observe que l'erreur faite par notre réseau semble croître quasi linéairement avec le bruit pour des paramètres d'entraînement équivalents. Il reste alors à définir un "seuil" de loss à partir duquel on considère les résultats obtenus comme satisfaisants.

Si pour notre cas on considère qu'au delà des 10% de loss le résultat est trop mauvais alors au delà d'un bruit de 0,5 notre modèle est trop peu performant.



Toujours dans l'optique de tester les performances de généralisation de notre auto-encodeur, on reprends l'entraînement de base réalisé sur une base bruitée à 0,5. On infère ensuite notre modèle avec des images bruitées à 0,2 et 0,8 et on obtient les résultats visuels suivants:

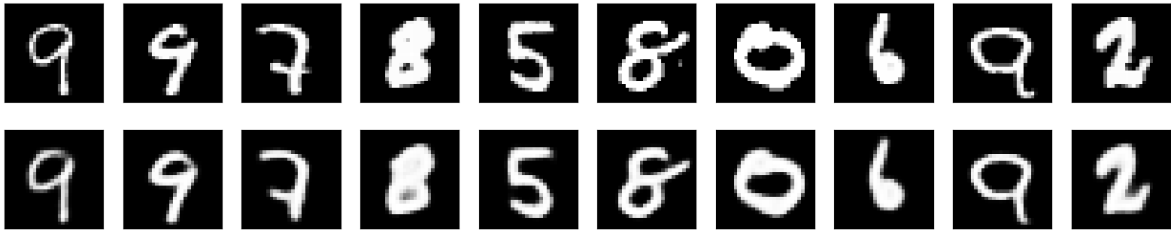


Figure 6: Inférence avec des images de niveau de bruit plus faible que la base d'entraînement (0,2 contre 0,5)

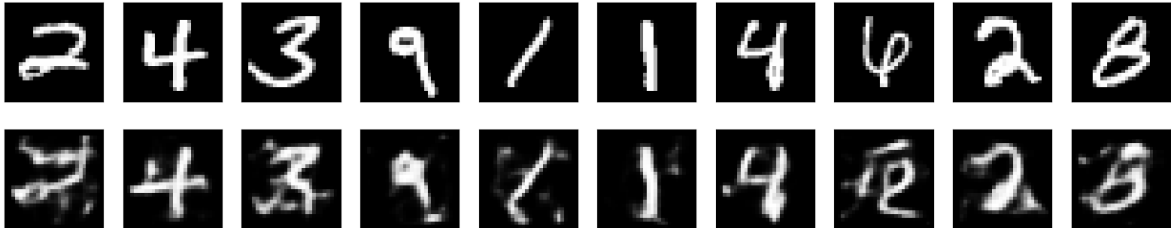


Figure 7: Inférence avec des images de niveau de bruit plus faible que la base d'entraînement (0,8 contre 0,5)

On constate que pour un niveau de bruit plus faible les résultats sont satisfaisant alors que lorsque le bruit devient plus grand que celui rencontré lors de l'entrainement, le modèle ne parvient plus à recréer des chiffres nets.

Cela met légèrement en évidence le coté data-spécifique des auto-encodeurs mais nous allons nous rendre compte de cet aspect plus en profondeur sur la prochaine partie

3 Application de notre auto-encodeur à une image quelconque

Par simplicité de mise en place on teste tout d'abord notre débruiteur en zero-shot sur des images de la base de donnée Fashion MNIST.

Avec cet exemple on constate bien l'aspect donnée-spécifique d'un auto-encodeur. En effet puisque ce dernier s'est entrainer à extraire des caractéristiques géométriques de chiffres il est très mauvais pour détecter autre chose qu'un chiffre.

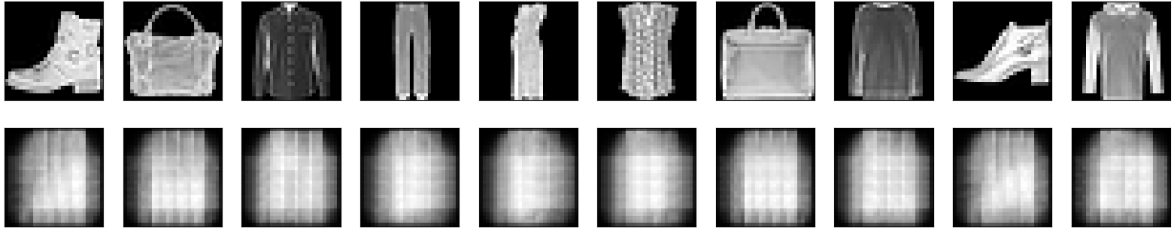


Figure 8: test en zéro-shot du débruiteur sur Fashion MNIST

Notre objectif final étant de pouvoir débruiter n'importe quelle image de dimensions quelconque on effectue également un test en zéro-shot sur l'image de léna que l'on découpe préalablement en patches de dimensions 28x28 puis que l'on recompose à l'aide de numpy et openCV. Encore une fois les résultats en zéro-shot sont très mauvais étant donné qu'on ne tente pas de débruiter des images de chiffres.

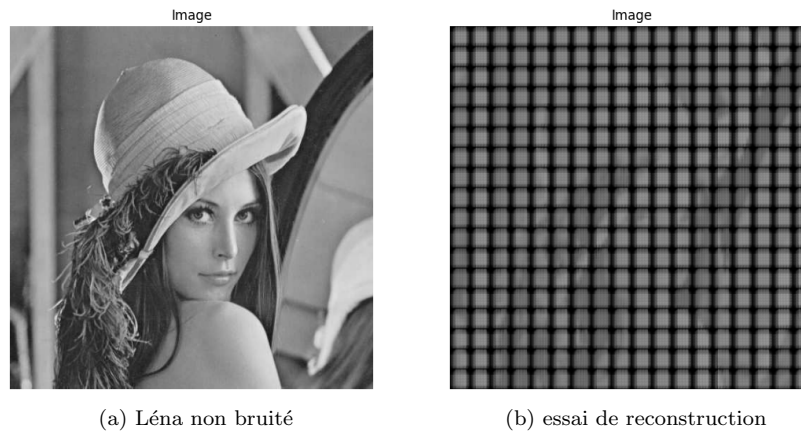


Figure 9: Test en zéro-shot de reconstruction de léna

En fait, on se rend compte que si l'entraînement se fait sur des images qui ont les mêmes caractéristiques (comme une bordure noire par exemple), on retrouvera ces mêmes artefacts sur tous les patches de la même image.

Nous faisons donc un entraînement directement sur des patches qui proviennent de la découpe de notre photo initiale :



Figure 10: Test avec entraînement sur des patches provenant de l'image

L'auto-encodeur fonctionne bien mieux, mais il a encore du mal avec les bords des images. Ceci est peut-être dû à un mauvais traitement des cas de bord, littéralement, à cause du "zero-padding" en bord

d'image pour l'étape de convolution ; pourtant, cette variable n'a eu aucun effet sur le résultat. Une autre idée était donc tout simplement d'augmenter la taille de la base de données d'entraînement en faisant des découpes selon des offsets différents de l'image, et cela semble avoir solutionné le problème.

Avec cette approche, on peut aussi réaliser une certaine quantité de débruitage (denoising) :

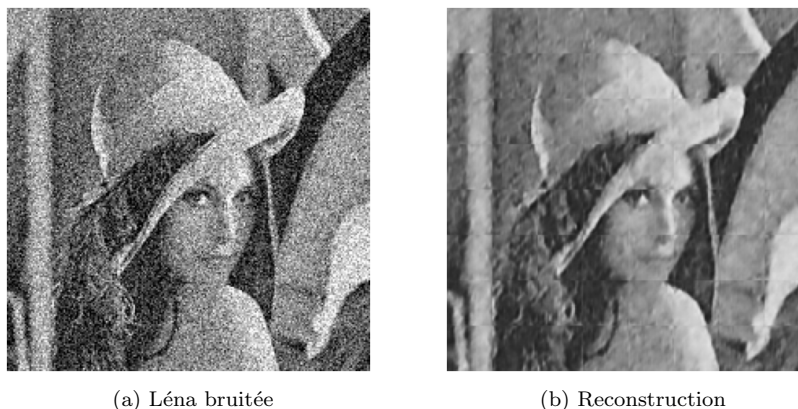


Figure 11: Résultat de l'auto-encodeur sur image bruitée

Et en empilant des prédictions de l'auto-encodeur avec deux offsets différents, on peut totalement éliminer cet effet de grille :

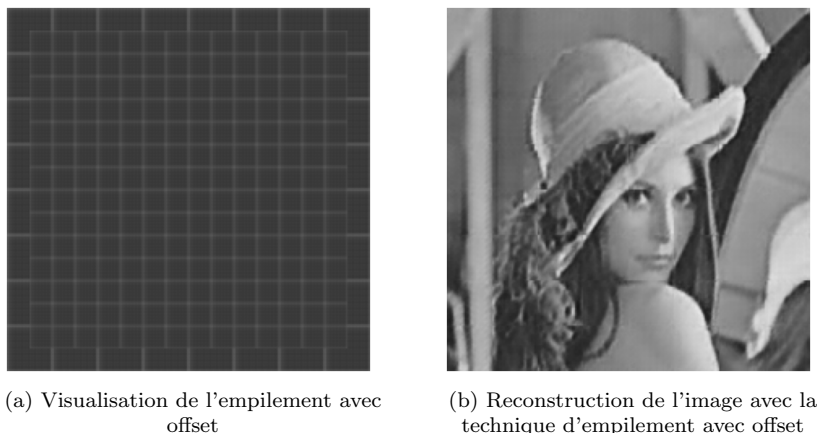


Figure 12: Schéma du stacking et résultat de cette technique

4 Application de notre auto-encodeur sur de grandes photos

Sur des photos bien plus grandes (2048×2048 plutôt que 255×255), l'auto-encodeur fonctionne bien, car il y a peu de détails à encoder. Ainsi, on peut porter une attention plus particulière au facteur de compression.

On se rend compte que l'auto-encodeur fourni ne compressait pas les images. En fait, le goulot d'étranglement était plus grand que l'entrée et que la sortie. En effet, le goulot contient $7 \times 7 \times 32 = 1568$ valeurs (soit environ 1,5 kB d'informations si encodé en float), alors qu'un patch ne représente que $28 \times 28 \times 1 = 784$ valeurs (environ 750 B). En ajoutant une couche de Conv2D et une de MaxPooling, on réussit à obtenir un facteur de compression de 2, ce qui est quatre fois meilleur qu'avant. Ceci pourrait permettre de stocker les images de grand format et, de manière générale, la sortie est assez indiscernable de l'entrée pour que cela ne soit pas trop gênant.



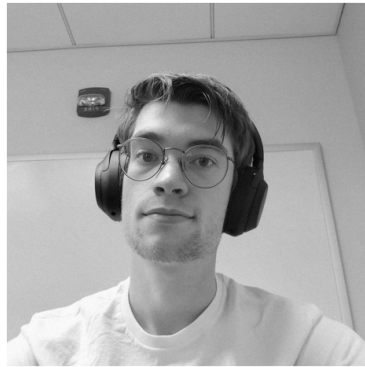
(a) Image de test



(b) Reconstruction avec artefacts

Figure 13: Illustration des artefacts créés sur une très grande image

Ces images ont été générées en utilisant l'auto-encodeur décrit plus haut et sont parfaitement utilisables. On observe aussi que le modèle conserve des capacités rudimentaires de débruitage.



(a) Image de test avec un bruit de 0.05



(b) Reconstruction avec stacking

Figure 14: Résultat de la méthode de stacking sur une grande image